

Machine Learning 2 – Final Project

Ariel Kreisberg Nitzav, Dvir Lafer

Introduction - Project Idea

The purpose of our project is to generate image captions using a deep learning algorithm.

Image captioning application takes an image as an input and produces a short textual summary (caption) that describes the content of the image. Our project is based on the paper [Show, Attend and Tell](#) by Kelvin Xu, Jimmy Lei Ba et al. (2016), which introduced the idea of attention to image captioning models.

We used the Flickr8k dataset which consists of images and their respective textual description, written as short, informative sentences - five sentences (captions) for each image.

Basic Part - Model 1

Model and algorithms

Our basic model consists of several modules:

1. FeatureExtractor.py - A module that utilizes the pre-trained InceptionV3 model to extract features from images. The module does that by truncating the prediction layer (*softmax*) and outputting the features which are used for later predictions.
2. DenseEncoder.py - A model that consists of one fully connected (FC) layer which allows the algorithm to learn a subset of features extracted from the trained InceptionV3.

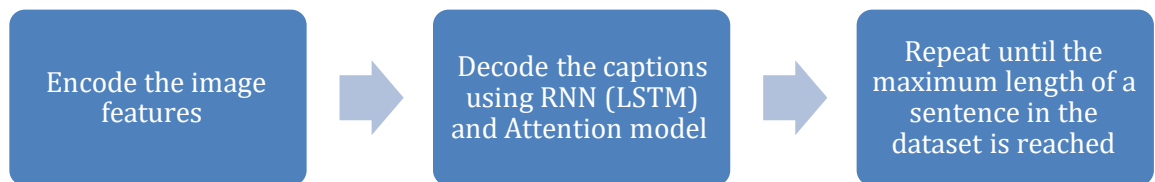
3. AttentionModel.py - A model that consists of two fully connected layers and one prediction layer. The attention module call function receives features of an image in the form of a tensor and a hidden state matrix produced by a LSTM.

After the expansion of the dimension of the hidden state to simulate a new iteration, one FC layer learns the features, the second learns the hidden state and the summation of both outputs is passed through a *tanh* activation function. This sequence is referred to as the “attention hidden layer”. Then, the prediction layer scores each feature, and its output is normalized using the *softmax* function which yields the attention weights. Using those weights, we create the context vector that is then used by the RNN decoder.

4. RNNDecoder.py - This model consists of an embedding layer, LSTM, two FC layers and the attention model described above. After creating the context vector as

described in the previous section, the tokenized caption is represented by its respective word embeddings, concatenated to the context vector. Then, it is passed through the LSTM layer. Afterwards, the returned hidden sequence is fed through two FC layers and the output is returned along with the current state of the LSTM layer.

5. ModelRun.py - A wrapper module for running the entire basic model, from start to finish. We had to create a custom training function (which utilizes a custom loss function and “train step” function) because the model consisted of several different stages in the process of generating a caption given an image. A rough scheme of the basic model:



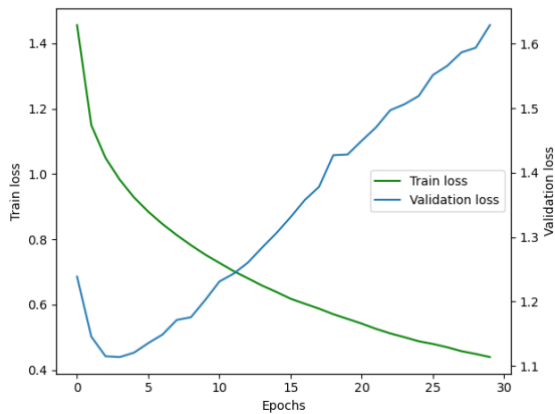
In addition, we created two more modules dedicated to the pre-processing of the dataset:

1. CaptionProcessor.py, which parses, cleans, and processes the captions into a manageable data container for later use.
2. ImageFeaturesLoader.py which extracts features from the images prior to the model’s training phase.

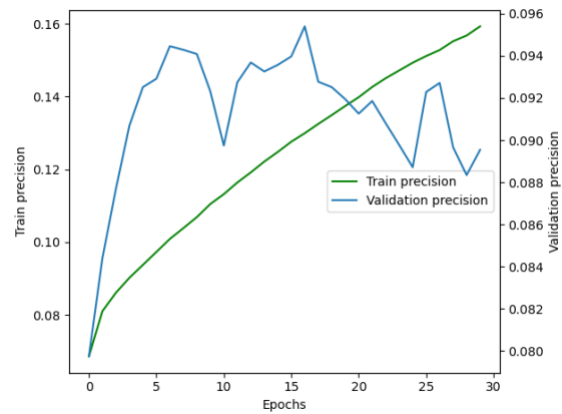
Training phase

During the training of the model, a training set and validation set were considered and used. The chosen loss function was Categorical Cross Entropy, and the accuracy metric that was chosen was Precision. The most common and accepted metric for this kind of tasks is the BLEU metric (Kishore Papineni, et al., 2002) which will be discussed later in this report, but because this metric is not built into Keras and in order not to interrupt the parallelization of the pipeline, we decided to use the precision function that was already built into the Keras module. The results of the training phase are as follows:

Loss - Train and Validation



Precision - Train and Validation



These graphs allow us to choose our models' weights reasonably. Hence, the weights from epoch 5 were picked, because the precision of our model performing on the validation set was relatively high in that epoch and the respective loss function has just started to increase slightly. This is to try to avoid overfitting on the training set.

Evaluation

As explained in the project proposal, the BLEU metric was chosen to evaluate how well our model was able to generate a reliable caption in comparison to a real one, produced by a human. The number attached to each BLEU metric reflects the n-gram overlap between a real caption and a generated caption in the metric calculation. Each of the BLEU scores were calculated on a test set that consisted of the same set of images, but in three variations: (1) The original images; (2) the same images, in grayscale, and (3) colorized versions of the same images that were produced using the model from the advanced part. This way, it was easier to tell how well or worse the models performed on different formats of images.

The average BLEU scores on the test set for the basic model that was trained only using the original (in color) images in the dataset appear under the "Evaluation" section in the advanced part.

Advanced Part

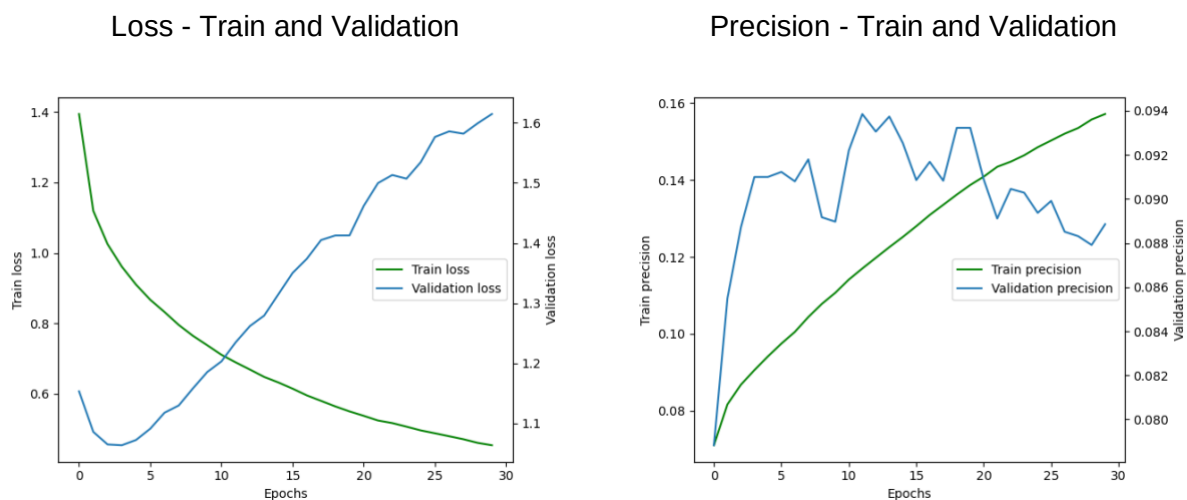
Gray images do not appear in the Flickr8k dataset, despite of being common in real life scenarios: satellite imaging, old family images, infrared imaging, etc. We wanted to be able to caption them using our model as well, and an approach we wanted to consider and test was whether **colorizing** gray images using a specialized model can improve our basic model's performance when handling this format of input.

Model and algorithms

The colorization model consists of one module, `ColorizationModel.py`. It utilizes an auto-encoder architecture: the first 19 layers of the pretrained VGG16 model used for object classification, and then alternates between 5 up-sampling layers and 6 convolutional layers with *ReLU* activation, whereas the last one is *tanh* activated. In order to learn to colorize, when training, each image is converted to $L^*a^*b^*$ colorspace, which contains three channels: L channel that tells how bright each pixel in the image is (this channel alone forms a grayscale image), and two channels, a and b, which together tell which hue the pixel is - thus, the prediction should be the a and b channels for each pixel in the image, because the L channel is already given as a part of the grayscale image. Then the L channel of each of the training images is fed to the network as the training data, and the respective a, b channels as targets. The model ran for 16 epochs and stopped after the validation set's accuracy converged to ~ 0.6 .

To assess how well colorizing gray images improves their captioning process, we first trained our captioning model using the colored training set used in the basic part alongside its grayscale version. Then, after colorizing the whole training set from gray, we used it and the original-colored images as a training set for a separate model and calculated the BLEU scores for each of the models.

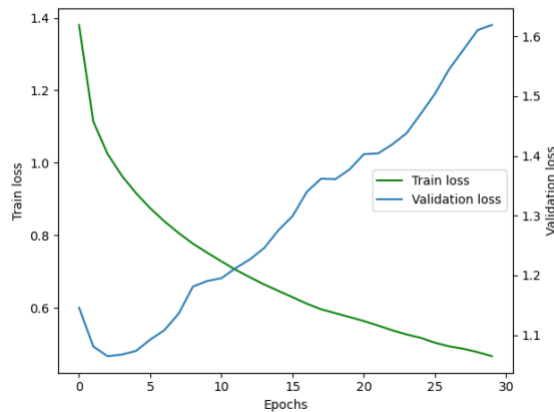
Model 2: Trained on a set that contains the original images in color, alongside their grayscale versions



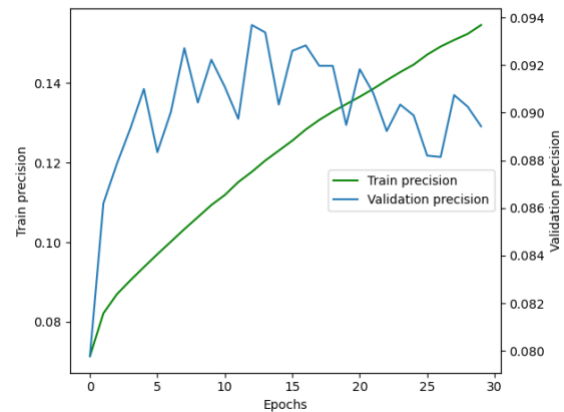
According to the precision of the validation we chose the weights of the 13th epoch.

Model 3: Trained on a set that contains the original images in color, alongside their colorized versions

Loss - Train and Validation



Precision - Train and Validation



According to the precision of the validation we chose the weights of the 14th epoch.

Evaluation

As mentioned above, the models are numbered as follows:

- 1- Trained on a set that contains the original images in color;
- 2- Trained on a set that contains the original images in color, alongside their grayscale versions;
- 3- Trained on a set that contains the original images in color, alongside their colorized versions

The following table summarizes the different BLEU scores for each model, when performing on a test set that solely contains either the images in their original form in color, their colorized or the grayscale version.

Each marked cell represents the best performance of a model given a BLEU score and a type of test set. For example, model 2 received the best BLEU2 score on grayscale images (0.531), hence it is marked and bolded.

	BLEU1			BLEU2			BLEU3			BLEU4		
Model	1	2	3	1	2	3	1	2	3	1	2	3
Original Images	0.767	0.749	0.766	0.536	0.521	0.532	0.321	0.313	0.318	0.217	0.213	0.216
Colorized Images	0.778	0.757	0.764	0.531	0.515	0.525	0.299	0.295	0.31	0.193	0.195	0.21
Gray Images	0.764	0.765	0.756	0.527	0.531	0.525	0.306	0.313	0.31	0.203	0.21	0.206

Comparing the results above, it appears that the performance of the basic model (model 1) on both grayscale and colorized images is comparable to its performance on regular, colored images. Specifically, its performance on grayscale wasn't as poor as we expected.

Moreover, colorizing the images wasn't proven as beneficial as we expected, as the performance of the grey model (model 2) on grey images was slightly better than the colorized model (model 3) on colorized images for each of the BLEU scores. In addition, the original model (model 1) performed better than the colorized model (model 3) for almost every BLEU score (except for BLEU4).

Possible explanations:

1. The colorization results were poor and inaccurate and did not add meaningful information to the caption generator model in the good case, and even "confused" the captioning model in the bad case. From the colorization examples in the appendix, it appears that the model managed to learn to correctly colorize vegetation, sky, and water, but almost everything else (people, animals, objects etc.) either left colorless or painted in dull sepia tone.
2. It is possible that the well-trained encoder that was used was able to extract effective features from both color and grayscale images, which reduced the difference in performance on the two.

In conclusion, in the current condition, the colorization process wasn't needed, assuming that the performance metrics of the basic model on grayscale images are bounded by the performance metrics on the original-colored images. That, in addition to the fact that the

basic model's performance on grayscale images (on which it wasn't trained) was comparable and almost equal to its performance on regular images.

Creative Part

After examining the sentences that our models created in the basic and advanced parts, we noticed that many of them were incomprehensible and did not make any sense, let alone follow grammar rules.

In a way, the model generated the captions by attaching words that it found best to describe each part of the image (using the attention model), while giving limited emphasis on the relation between the words themselves, and on whether they form any logical sense together. This observation led us to look for ideas for "forcing" the basic model's generator to create more meaningful and realistic sentences.

Before reaching the chosen approach, we explored several methods such as:

- Giving more freedom to the generator by minimizing the pre-processing stage – during the pre-process stage we eliminated characters like punctuation and single characters words (e.g., 'a'), which otherwise would have been beneficial in rendering a realistic caption. Eventually, we decided to keep the character elimination as it was after learning that their inclusion could most probably "confuse" the model, rather than lead it to better pattern detection and better captioning.
- Basic rule based "brute force" grammatic model that enforces grammatic rules on the caption generator - This approach interferes with the training process and forces the generator to follow some basic grammar rules. In our case, this approach was merely theoretical because it is almost impossible to enumerate correct grammatic structures for the model to follow, let alone combine it with the training procedure.
- Grammar correction models - this was our preferred approach, but after going through numerous articles about it and consulting with Uri, we realized that we didn't have the time and the capacity for creating a real, functioning, grammar correction model that could improve our initial results.

Eventually, inspired by the idea of making the sentences more realistic, we came up with the idea of using a GAN-aided approach. In a GAN, a discriminator and a generator "compete" against each other – the first tries to tell apart real data (images, text) from data generated by the generator, while the latter tries to produce better output to "fool" the discriminator into classifying it as the real one.

Model and algorithms

The use of the GAN-aided approach allowed us to use our former generator model and implement a new discriminator model along with a new custom training process:

1. `Discriminator.py` - This model is inspired by the RNN decoder model procedures. At each call, the model utilizes an embedding layer to embed the given captions. Then, it sums the image's features and expands their dimension to fit them with the dimension of the respective caption. Afterwards the two are concatenated and fed through an LSTM layer. The LSTM layer returns the last hidden state and the discriminator uses a FC layer to reduce the dimensions of the state to 1, allowing the discriminator to use a *sigmoid* function and predict whether the given captions were real (i.e., from the dataset) or "fake" (generated by the generator).
2. `ModelRunGAN.py` - This module follows most of the procedures from the basic model's `ModelRun.py` but with custom training step and loss objectives.

It deals with two loss objectives: the discriminator's loss, which is calculated based on how well it predicted whether the inputs were real or "fake" using two binary cross entropy functions, one for the real captions and one for the "fake", generated captions.

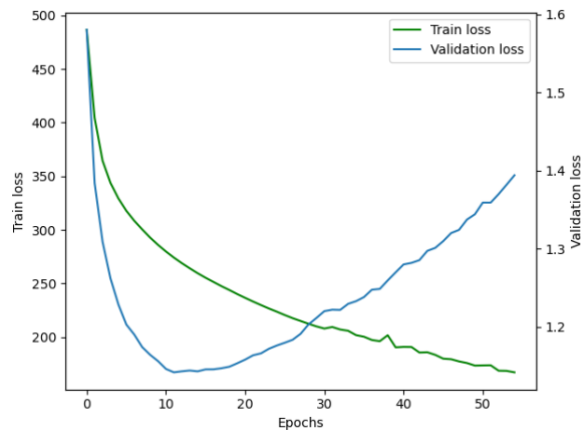
The generator's loss is composed of two loss objectives:

- Sparse categorical entropy that checks the proximity between the generated caption and the real caption.
- Inverse sum of the discriminator's predictions on the generated captions, to "punish" the generator when classifying fake captions as real ones.

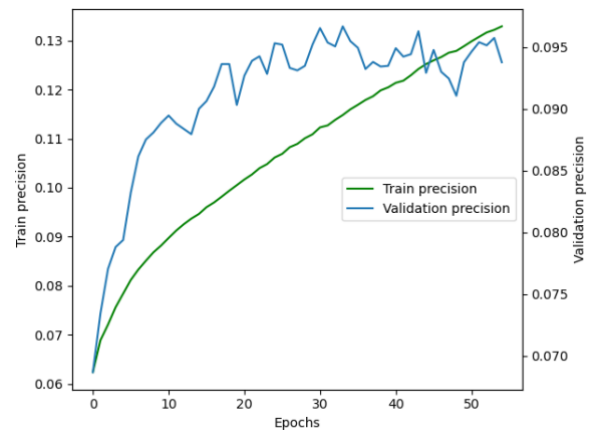
Summing those two objectives allowed us to keep the network trainable (using plain binary cross entropy function did not result in valid gradients because in that setting, it wasn't differentiable). This, while "rewarding" the generator with a higher reward when it managed to fool the discriminator.

Model 4: Trained on a set that contains only the original images in color, using GAN-aided approach

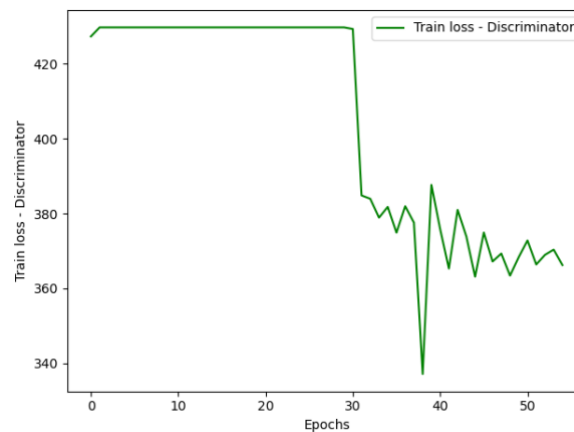
Loss (Generator) - Train and Validation



Precision (Generator) - Train and Validation



Loss (Discriminator) - Train and Validation



Because we are using only the generator's weights during the BLEU scores evaluation, we aimed to choose the best weights with respect to the validation accuracy and loss. Even though the validation loss was very high at the 50th epoch, it seemed that the validation accuracy did not decrease and was even close to its highest score. We also took into consideration the discriminator's loss which was at the second lowest point at the 50th epoch. Concluding from all the above reasons, we used the 50th epoch's weights to evaluate the BLEU scores.

Evaluation

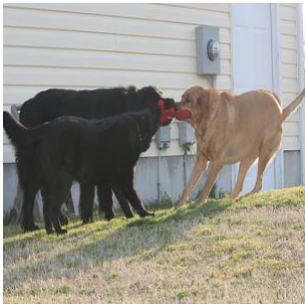
The average BLEU scores for the GAN model are as follows, with the original model performance (model 1) in parenthesis:

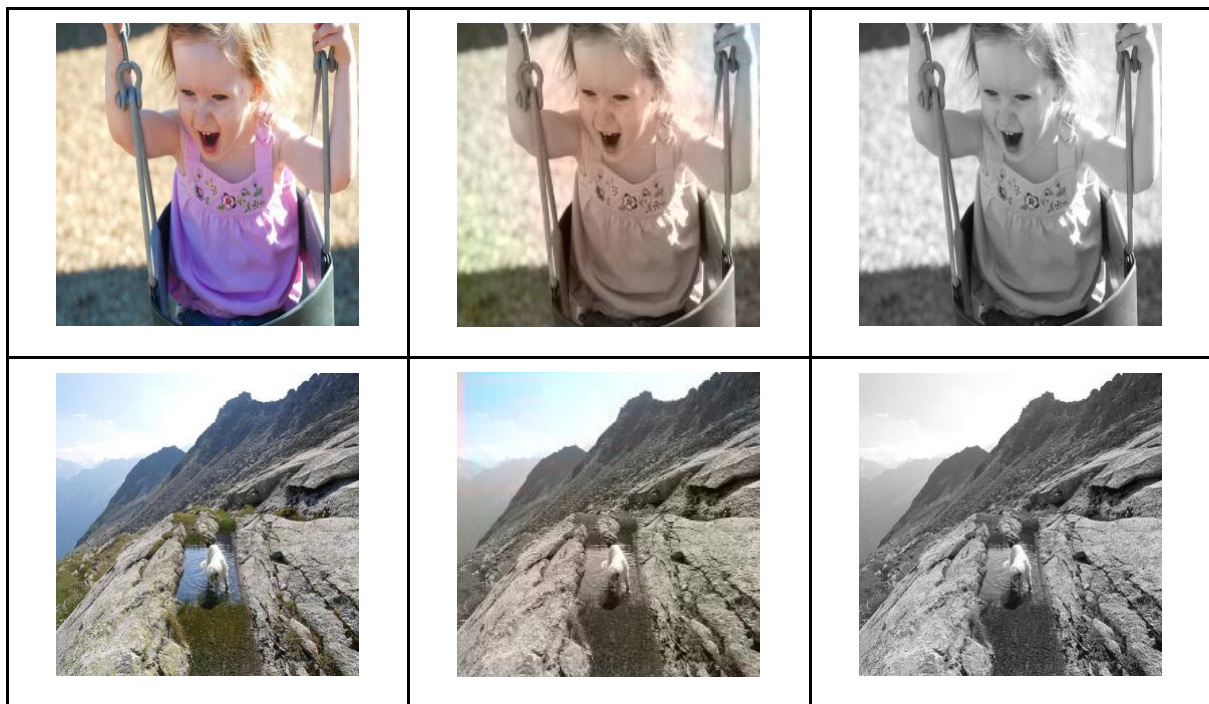
Model 4	BLEU1	BLEU2	BLEU3	BLEU4
Original Images	0.752 (0.767)	0.538 (0.536)	0.339 (0.321)	0.24 (0.217)

The results ended up in what may seem like only a slight improvement but considering the fact that the BLEU4 (4-gram BLEU) score was improved by ~0.023 on average, this is an indication that the generator managed to generate more comprehensible sentences. This is because improving the 4-gram results means that the model generates longer word sequences more accurately – a very important aspect in generating more naturalistic captions. In the appendix there are several examples for sentences generated with the GAN architecture and without it.

Appendix

Sample images, original vs. colorized vs. grayscale version

Original Image	Colorized Image	Gray Image
		
		
		
		



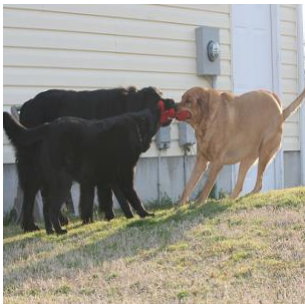
Examples for captions generated by model 1 given different formats of the same images

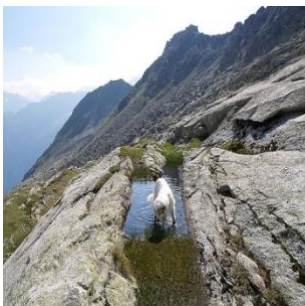
Image	Sample of an original caption	Predicted caption
	A dog is jumping over a stream of water on a sandy beach	Original Image: Wet hairy dog is jumping on blue sky at the water white waves on dirt shoreline
		Colorized Image: The dog tries to catch the background
		Gray Image: Dog is running and yellow top runs across the wet baseball getting ready to put
	Brown dog and two black dogs have three-way tug on	Original Image: Black dog is wrestling

	red object, outdoors	Colorized Image: The dog with backpack is staring at the camera
		Gray Image: Dog lays on the clouds
	A boy in a red swimsuit jumps into the water to join two people	Original Image: Two people are playing in the water
		Colorized Image: Man on his feet in the water
		Gray Image: Man in green trunks is surfing pick in the background
	A bearded man is sitting on a bench wearing a fur lined jacket and hat	Original Image: Boy looking at the road
		Colorized Image: Woman holding flower smiles beside the middle of bronze dolly
		Gray Image: An asian woman dressed like shine poling his shoes
	A little girl on a kid swing	Original Image: Boy is playing on the camera
		Colorized Image: Two young baby around
		Gray Image: The child hangs onto horse

	A white dog standing in a pool of water high in the mountains	Original Image: Rock climbing top of an icy terrain
		Colorized Image: Person wearing an orange shirt is abseiling down cliff drift
		Gray Image: An older couple wearing backpack stands on top of some mountain walls

Basic model (model 1) vs. GAN-aided model

Image	Sample of an original caption	Predicted caption
	A dog is jumping over a stream of water on a sandy beach	Original: Wet hairy dog is jumping on blue sky at the water white waves on dirt shoreline
		GAN-aided: brown and white dog holding the rocks
	Brown dog and two black dogs have three-way tug on red object, outdoors	Original: Black dog is wrestling
		GAN-aided: black dog stands in front of red toy in its paw with red brown dog in harness over beautiful orange black on the sand

	A boy in a red swimsuit jumps into the water to join two people	Original: Two people are playing in the water
		GAN-aided: the man in swimming
	A bearded man is sitting on a bench wearing a fur lined jacket and hat	Original: Boy looking at the road
		GAN-aided: the man wearing jacket as she is wearing hat is standing
	A little girl on a kid swing	Original: Boy is playing on the camera
		GAN-aided: the guy in green shirt holds onto something
	A white dog standing in a pool of water high in the mountains	Original: Rock climbing top of an icy terrain
		GAN-aided: man climbing rock

More colorized pictures examples

